

Thesis: A Comparative Study of Large Language Models for Financial Sentiment Analysis and Their Predictive Potential for Short-Term Stock Price Movement

Component: Experiment 2 - Model B: Meta Llama 3 (8B Instruct)

Description: This notebook loads the prepared 1000 headline dataset and evaluates Llama 3 8B on Experiment 2 using zero-shot prompting. It maps sentiment predictions to directional signals and computes directional accuracy, precision, recall, and F1-score based on next-day stock price movement.

Select T4 GPU as runtime.

```
# Install bitsandbytes – restart required after this

!pip install -q -U bitsandbytes accelerate
```

Go to runtime -> restart this session again -> then run cell 1 and run cell 2

```
# Import required libraries – Experiment 2b: Llama 3

import pandas as pd
import numpy as np
import torch
import warnings
import gc

from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report
)

warnings.filterwarnings("ignore")

print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU detected      : {torch.cuda.get_device_name(0)}")
```

```
PyTorch version : 2.10.0+cu128
CUDA available   : True
GPU detected     : Tesla T4
```

Add the API Key from Hugging Face using "Add New Secret" in Google Colab

```
# Connect to Hugging Face and load Experiment 2 dataset

from google.colab import userdata, drive
from huggingface_hub import login

drive.mount("/content/drive")

hf_token = userdata.get("HF_TOKEN")
login(token=hf_token)

df_exp2 = pd.read_csv("/content/drive/MyDrive/Thesis_Data/exp2_dataset.csv")

print("Dataset loaded.")
print(f"Total headlines : {len(df_exp2)}")
print(f"\nMovement distribution:")
print(df_exp2["movement"].value_counts())
```

```
Mounted at /content/drive
Dataset loaded.
Total headlines : 1000
```

```
Movement distribution:
movement
1      508
0      492
Name: count, dtype: int64
```

```
# Load Llama 3 8B with 4-bit quantisation

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
```

```

        bnb_4bit_compute_dtype=torch.float16,
        bnb_4bit_use_double_quant=True
    )

    print("Loading Llama 3 8B...")

    llama_tokenizer = AutoTokenizer.from_pretrained(
        "meta-llama/Meta-Llama-3-8B-Instruct",
        token=hf_token
    )

    llama_model = AutoModelForCausalLM.from_pretrained(
        "meta-llama/Meta-Llama-3-8B-Instruct",
        quantization_config=bnb_config,
        device_map="auto",
        token=hf_token
    )

    print("Llama 3 8B loaded successfully.")

```

```

Loading Llama 3 8B...
config.json: 100% 654/654 [00:00<00:00, 25.9kB/s]

tokenizer_config.json: 51.0k/? [00:00<00:00, 660kB/s]

tokenizer.json: 9.09M/? [00:01<00:00, 6.70MB/s]

special_tokens_map.json: 100% 73.0/73.0 [00:00<00:00, 1.22kB/s]

model.safetensors.index.json: 23.9k/? [00:00<00:00, 1.55MB/s]

Download complete: 100% 16.1G/16.1G [02:29<00:00, 177MB/s]

Fetching 4 files: 100% 4/4 [02:29<00:00, 149.06s/it]

Loading weights: 100% 291/291 [01:05<00:00, 158.52it/s, Materializing param=model.norm.weight]

generation_config.json: 100% 187/187 [00:00<00:00, 17.3kB/s]

Llama 3 8B loaded successfully.

```

```

def classify_llama3(text):
    messages = [
        {"role": "system", "content": "You are a financial sentiment classifier. Classify the sentiment as exact"},
        {"role": "user", "content": f"Classify the sentiment of this financial text:\n\nText: {text}\n\nRespond"}
    ]

    tokenized = llama_tokenizer.apply_chat_template(
        messages,
        add_generation_prompt=True,
        return_tensors="pt",
        return_dict=True
    ).to(llama_model.device)

    with torch.no_grad():
        outputs = llama_model.generate(
            *tokenized,
            max_new_tokens=5,
            do_sample=False,
            pad_token_id=llama_tokenizer.eos_token_id
        )

    input_length = tokenized["input_ids"].shape[1]
    generated = llama_tokenizer.decode(
        outputs[0][input_length:],
        skip_special_tokens=True
    )
    # Default to positive if unclear
    label = generated.strip().lower()
    if "negative" in label:
        return "negative"
    else:
        return "positive"

    print("Llama 3 classifier ready.")

```

```
Llama 3 classifier ready.
```

```

# Run Llama 3 on 1,000 headlines dataset

headlines = df_exp2["headline"].tolist()
llama3_preds = []
total = len(headlines)

```

```

print(f"Running Llama 3 on {total} headlines...")
print("-" * 40)

for i, text in enumerate(headlines):
    label = classify_llama3(text)
    llama3_preds.append(label)

    if (i + 1) % 100 == 0:
        print(f" Progress: {i+1}/{total}")

print(f"\nDone. Total predictions: {len(llama3_preds)}")
print(f"\nSentiment distribution:")
print(pd.Series(llama3_preds).value_counts())

```

The following generation flags are not valid and may be ignored: ['temperature', 'top_p']. Set `TRANSFORMERS_VERB

Running Llama 3 on 1000 headlines...

```

-----
Progress: 100/1000
Progress: 200/1000
Progress: 300/1000
Progress: 400/1000
Progress: 500/1000
Progress: 600/1000
Progress: 700/1000
Progress: 800/1000
Progress: 900/1000
Progress: 1000/1000

```

Done. Total predictions: 1000

```

Sentiment distribution:
positive    580
negative    420
Name: count, dtype: int64

```

```

# Map sentiment to directional prediction and evaluate

df_exp2["llama3_sentiment"] = llama3_preds
df_exp2["llama3_direction"] = df_exp2["llama3_sentiment"].map({
    "positive": 1,
    "negative": 0
})

true_movement = df_exp2["movement"].tolist()
pred_movement = df_exp2["llama3_direction"].tolist()

acc = accuracy_score(true_movement, pred_movement)
prec = precision_score(true_movement, pred_movement, average="macro")
rec = recall_score(true_movement, pred_movement, average="macro")
f1 = f1_score(true_movement, pred_movement, average="macro")

print("=" * 50)
print("Llama 3 8B - Experiment 2 Results (next-day movement)")
print("=" * 50)
print(f" Directional Accuracy : {acc:.4f} ({acc*100:.2f}%)")
print(f" Precision              : {prec:.4f}")
print(f" Recall                  : {rec:.4f}")
print(f" F1-Score                : {f1:.4f}")
print("=" * 50)
print(classification_report(true_movement, pred_movement,
    target_names=["Down (0)", "Up (1)"]))

```

```

=====
Llama 3 8B - Experiment 2 Results (next-day movement)
=====

```

```

Directional Accuracy : 0.5000 (50.00%)
Precision             : 0.4987
Recall                : 0.4987
F1-Score              : 0.4961

```

```

=====
              precision    recall  f1-score   support

   Down (0)         0.49         0.42         0.45         492
    Up (1)          0.51         0.58         0.54         508

   accuracy                   0.50         1000
  macro avg              0.50         0.50         0.50         1000
 weighted avg              0.50         0.50         0.50         1000

```

```

# Save Llama 3 Experiment 2 results

```

```

llama3_exp2_scores = {
    "model": "Llama 3 8B",

```

```
"directional_accuracy": round(acc, 4),
"precision": round(prec, 4),
"recall": round(rec, 4),
"f1_score": round(f1, 4)
}

print("Llama 3 8B - Experiment 2 Results Summary")
print(pd.DataFrame([llama3_exp2_scores]))

# Save to Google Drive
df_exp2.to_csv("/content/drive/MyDrive/Thesis_Data/exp2_llama3_preds.csv", index=False)
print("\nSaved to Google Drive.")
```

```
Llama 3 8B - Experiment 2 Results Summary
      model  directional_accuracy  precision  recall  f1_score
0  Llama 3 8B                0.5      0.4987  0.4987    0.4961

Saved to Google Drive.
```